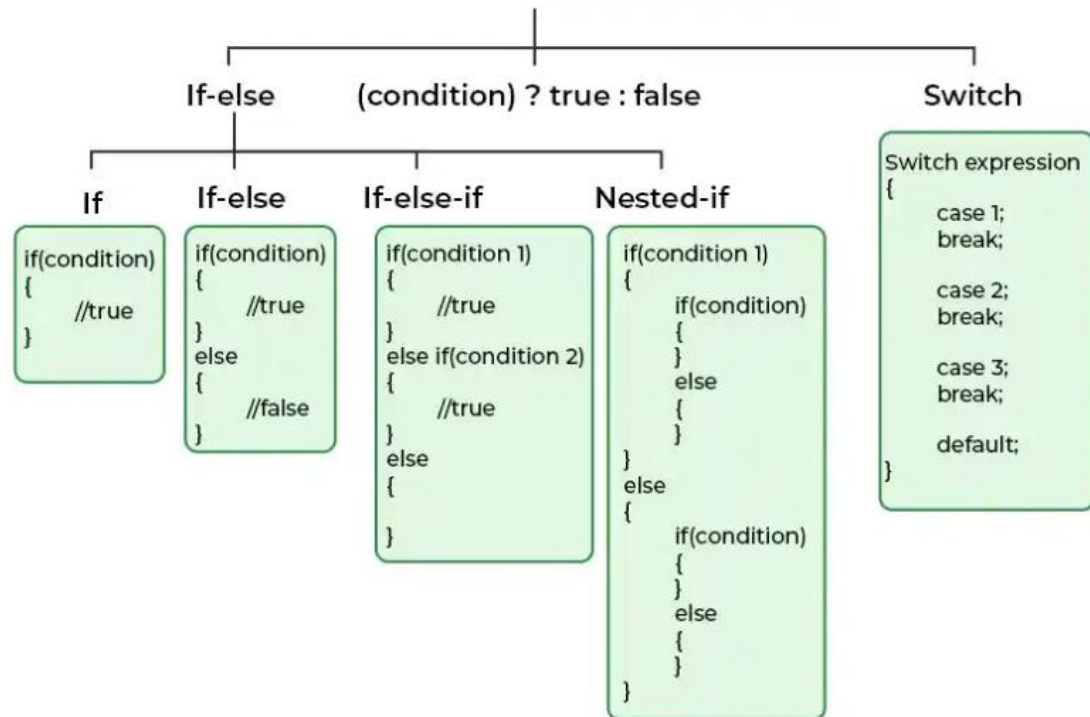


Control Structures

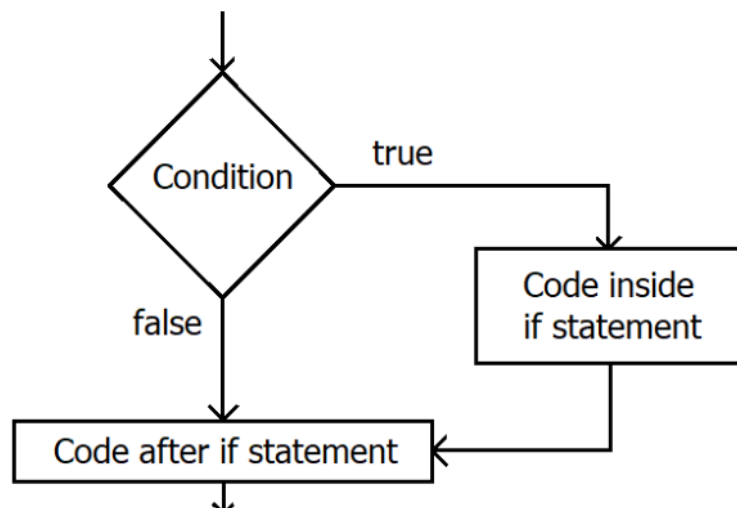
Conditional Statements in C



If statement

The if statement is the simplest decision-making statement in C. It executes a block of code only when the specified condition evaluates to true.

- If the condition is true, the statements inside the “if” block are executed.
- If the condition is false, the “if” block is skipped and execution continues with the next statement.



```
#include <stdio.h>
int main()
{
```

```
int age = 20;
if (age >= 18)
{
    printf("Eligible for vote");
}
return 0;
}
```

Eligible for vote

```
#include <stdio.h>
int main() {
    int n = 9;

    if (n < 10) {
        printf("%d < 10", n);
    }

    if (n > 20) {
        printf("%d > 20", n);
    }

    return 0;
}
```

9 < 10

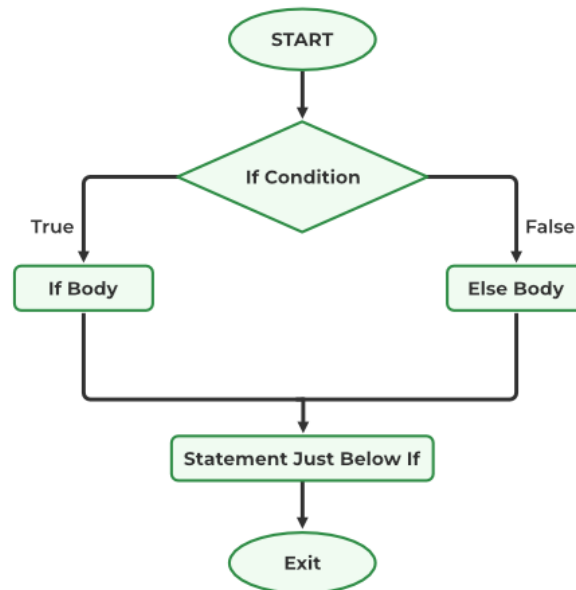
```
#include <stdio.h>
int main() {
    int n = 4;
    if (n % 2 == 0) {
        printf("%d is Even", n);
    }
    if (n % 2 != 0) {
        printf("%d is Odd", n);
    }
    return 0;
}
```

4 is Even

If-else statement

The if-else statement is used to execute one block of code if a condition is true and another block if it is false.

- The if block executes when the condition is true.
- The else block executes when the condition is false.



```

#include <stdio.h>
int main()
{
    int n = 10;
    if (n > 5)
    {
        printf("%d is greater than 5", n);
    }
    else
    {
        printf("%d is less than 5", n);
    }
    return 0;
}
10 is greater than 5
  
```

Negative Number check using If-else statement

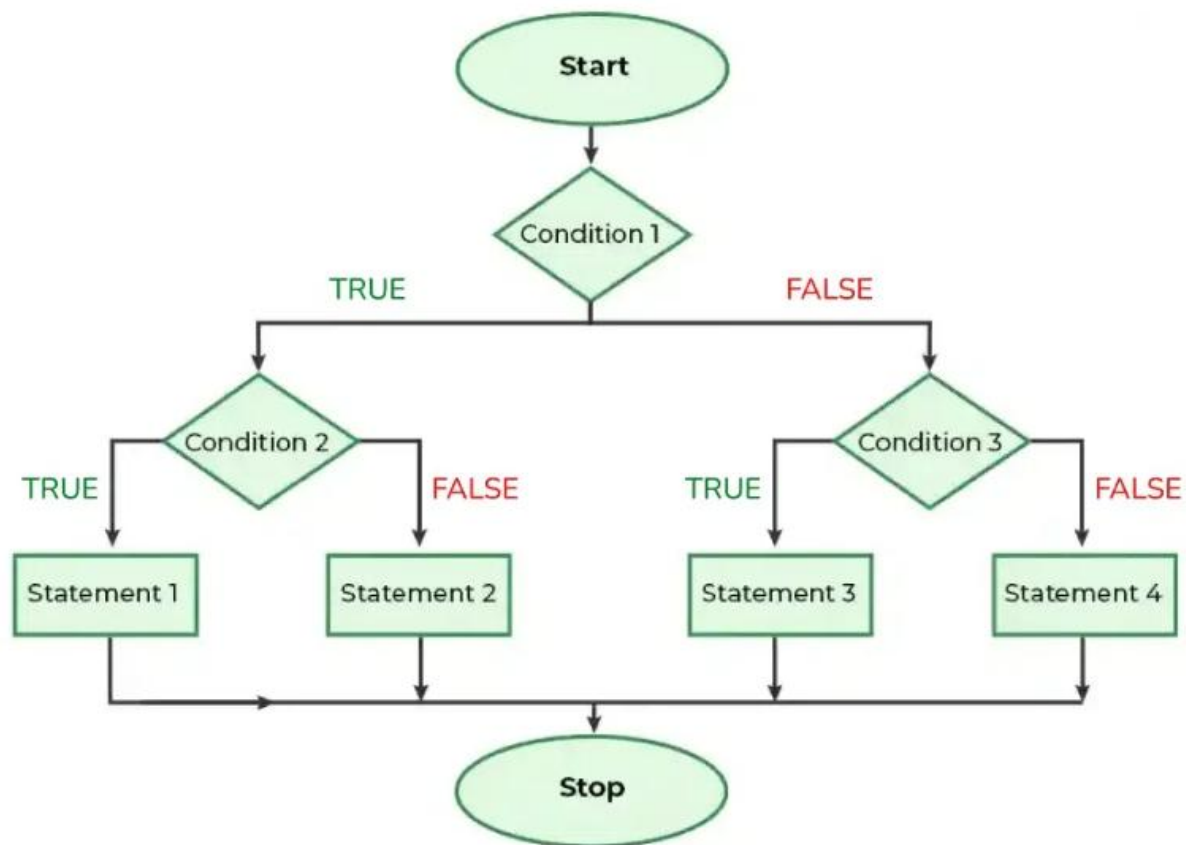
```

#include <stdio.h>
int main()
{
    int n = -7;
    if (n < 0)
        printf("Negative");
    else
        printf("Not Negative");
    return 0;
}
Negative
  
```

Nested if-else statement

A nested if-else is an if or if-else statement placed inside another if or else block.

- It is used to check multiple conditions in a step-by-step manner.
- The inner if statement is evaluated only if the outer condition allows it.



```

#include <stdio.h>
int main(){
    int age = 11;
    if (age >= 18)
    {
        if (age >= 60)
            printf("Eligible to vote (Senior Citizen)\n");
        else
            printf("Eligible for vote\n");
    }
    else
    {
        printf("Not eligible to vote (Under 18)\n");
        if (age >= 13)
            printf("teenager\n");
        else
            printf("not a teenager\n");
    }

    return 0;
}
  
```

Not eligible to vote (Under 18)
not a teenager

Largest Among Three Numbers Using If-else

```

#include <stdio.h>
int main()
{
  
```

```

int a = 1, b = 2, c = 11;
if (a >= b)
{
    if (a >= c)
        printf("%d", a);
    else
        printf("%d", c);
}
else
{
    if (b >= c)
        printf("%d", b);
    else
        printf("%d", c);
}

return 0;
}

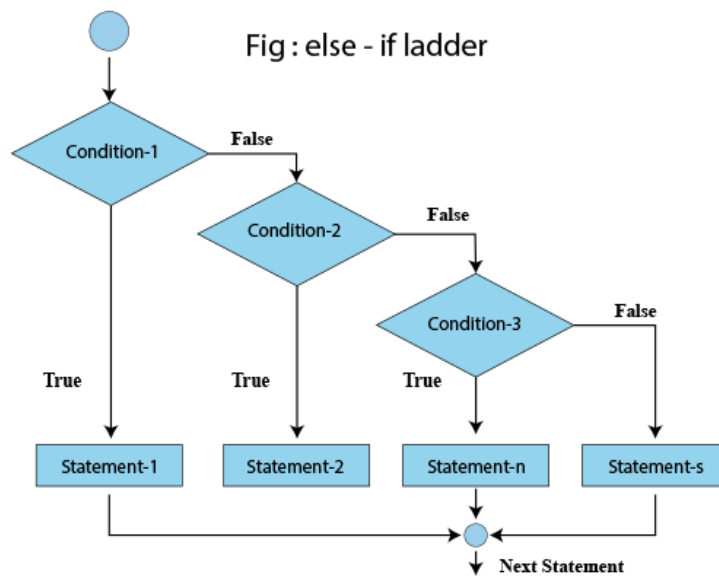
```

11

if-else-if Ladder statement

The if-else-if ladder is used to test multiple conditions and execute the block corresponding to the first true condition.

- Conditions are checked from top to bottom until one evaluates to true.
- If no condition is true, the else block is executed (if present).



```

#include <stdio.h>
int main() {
    int i = 20;
    if (i == 10)
        printf("Not Eligible");
    else if (i == 15)
        printf("wait for three years");
    else if (i == 20)

```

```
printf("You can vote");
else
printf("Not a valid age");
return 0;
}
```

[You can vote](#)

Check if the number is positive, negative, or zero

```
#include <stdio.h>
int main()
{
    int n = 10;
    printf("Number = %d \n", n);
    if (n > 0)
    {
        printf("Positive.\n");
    }
    else if (n < 0)
    {
        printf("Negative.\n");
    }
    else
    {
        printf("Zero.\n");
    }
    return 0;
}
```

[Number = 10](#)

[Positive.](#)

```
#include <stdio.h>
int main() {
    int mark;
    printf("Enter your mark: ");
    scanf("%d", &mark);

    if (mark >= 90)
        printf("Grade A");
    else if (mark >= 80)
        printf("Grade B");
    else if (mark >= 70)
        printf("Grade C");
    else if (mark >= 60)
        printf("Grade D");
    else if (mark >= 50)
        printf("Grade E");
    else
        printf("Grade F");

    return 0;
}
```

[Example 1](#)

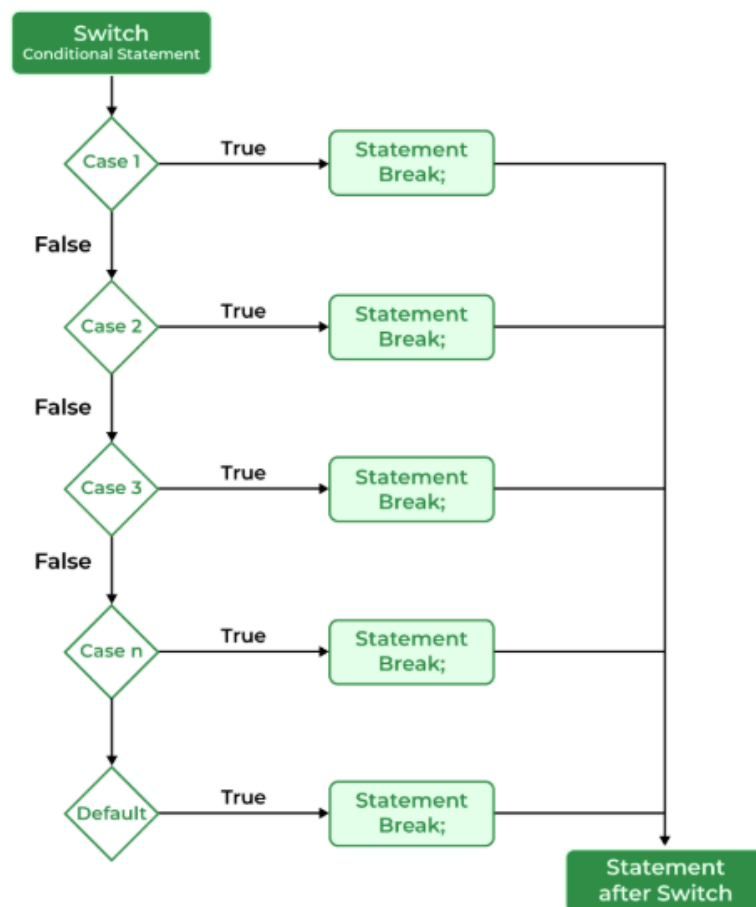
Enter your mark: 85
Grade B

Example 2
Enter your mark: 45
Grade F

Switch Statement

The switch case statement is used to execute different blocks of code based on the value of an expression or variable.

- It provides an alternative to the if-else-if ladder for multiple value-based conditions.
- The matching case is executed based on the value of the switch expression.



```
#include <stdio.h>
int main() {
    int var = 18;
    switch (var) {
        case 15:
            printf("You are a kid");
            break;
        case 18:
            printf("Eligible for vote");
```

```
        break;
    default:
        printf("Default Case is executed");
        break;
    }
    return 0;
}
```

Eligible for vote

C switch Statement without Break

```
#include <stdio.h>
int main() {
    int var = 2;

    // switch case without break
    switch (var) {
    case 1:
        printf("Case 1 is executed.\n");
    case 2:
        printf("Case 2 is executed.\n");
    case 3:
        printf("Case 3 is executed.\n");
    case 4:
        printf("Case 4 is executed.");
    }
    return 0;
}
```

Case 2 is executed.
Case 3 is executed.
Case 4 is executed.

Nested switch Statement

```
#include <stdio.h>
int main()
{
    int department = 1;
    int year = 2;
    switch (department)
    {
        case 1:
            printf("EEE Department\n");
            switch (year)
            {
                case 1:
                    printf("First Year\n");
                    break;
                case 2:
                    printf("Second Year\n");
                    break;
            }
            break;
        case 2:
```

```

printf("ECE Department\n");
switch (year)
{
    case 1:
        printf("First Year\n");
        break;

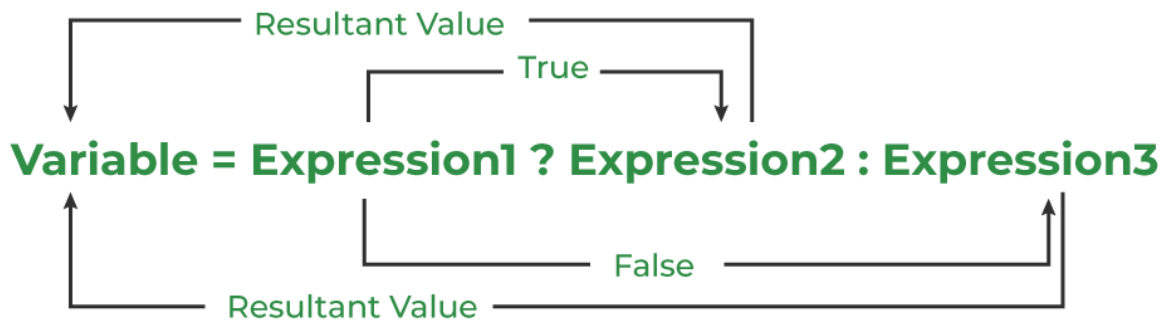
    case 2:
        printf("Second Year\n");
        break;
}
break;
}
return 0;
}
EEE Department
Second Year

```

Conditional or Ternary Operator (?:)

The conditional operator (?:) is used to choose between two expressions based on a condition.

- It is a shorthand alternative to the if-else statement.
- It works with three operands: a condition, a true expression, and a false expression.



```

#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int max = (a > b) ? a : b;
    printf("Maximum = %d", max);
    return 0;
}
Maximum = 20

```

```

#include <stdio.h>
int main()
{
    int m = 5, n = 4;
    (m > n) ? printf("m is greater than n that is %d > %d", m, n)
            : printf("n is greater than m that is %d > %d", n, m);
    return 0;
}

```

```

}
m is greater than n that is 5 > 4
#include <stdio.h>
int main()
{
    int a = 10, b = 20, c = 15;
    int big = (a > b) ?
        ((a > c) ? a : c) :
        ((b > c) ? b : c);

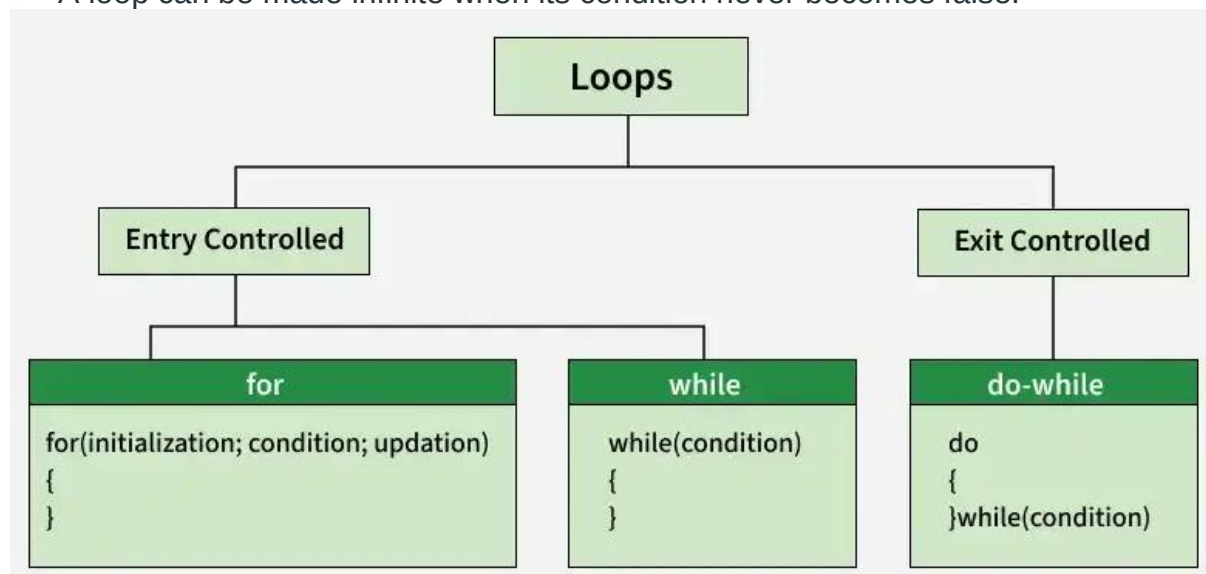
    printf("Biggest = %d", big);
    return 0;
}
Biggest = 20

```

Loops in C

Loops in C are used to repeatedly execute a block of code as long as a specified condition is satisfied

- C provides three main loops: for, while, and do-while.
- Loops can also be nested and can be controlled using statements such as break and continue.
- A loop can be made infinite when its condition never becomes false.



For loop

It is an entry-controlled loop, which means that the condition is checked before the loop's body executes.

```

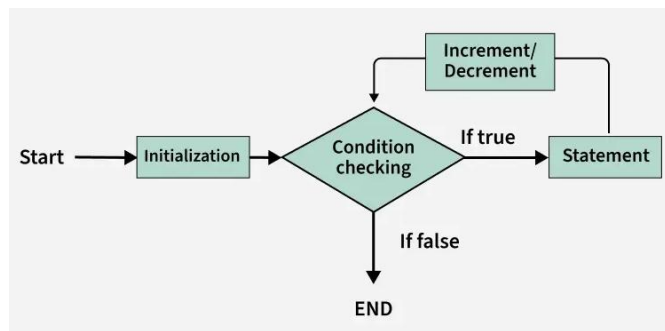
for (initialization; condition; update) {
    // Code block to be executed
}

```

The main parts of a for loop are:

- **Initialization:** Sets the initial value of the loop variable.
- **Condition:** Determines whether the loop should continue executing.
- **Update:** Changes the loop variable after each iteration.

- **Body:** Contains the statements to be executed repeatedly.



```
#include <stdio.h>
int main() {
    for (int i = 0; i < 5; i++) {
        printf( "%d ", i + 1);
    }
    return 0;
}
```

1 2 3 4 5

Nested for Loop

```
#include <stdio.h>
int main() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            printf("i = %d, j = %d\n", i, j);
        }
    }
    return 0;
}
```

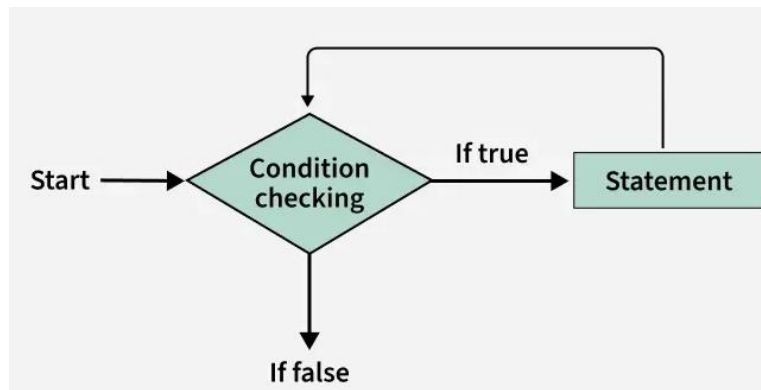
i = 0, j = 0
 i = 0, j = 1
 i = 1, j = 0
 i = 1, j = 1
 i = 2, j = 0
 i = 2, j = 1

While Loop

It is also an entry-controlled loop in which the condition is checked before entering the body.

```
while (condition) {
    // Body of loop }
```

The initialization and update of the loop variable are performed separately.



```

#include <stdio.h>
int main() {

    // Initialization expression
    int i = 0;

    // Test expression
    while(i <= 5) {
        printf("%d ", i + 1);

        // update expression
        i++;
    }
    return 0;
}
1 2 3 4 5 6
  
```

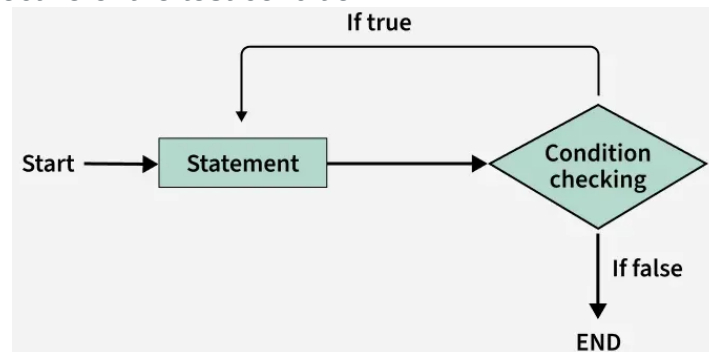
Nested while loop

```

#include <stdio.h>
int main()
{
    int i = 1, j;
    while (i <= 2)
    {
        j = 1;
        while (j <= 3)
        {
            printf("%d ", j);
            j++;
        }
        printf("\n");
        i++;
    }
    return 0;
}
1 2 3
1 2 3
  
```

Do-while Loop

The do-while loop is an exit-controlled loop, which means that the condition is checked after executing the loop body. Due to this, the loop body will execute at least once irrespective of the test condition.



```
do {  
    // Body of loop  
} while (condition);
```

```
#include <stdio.h>  
int main() {  
    int i = 0;  
    do  
    {  
        printf( "%d ", i);  
        i++;  
    } while (i <= 10);  
  
    return 0;  
}
```

0 1 2 3 4 5 6 7 8 9 10

Nested do-while Loop

A nested do-while loop means having one do-while loop inside another do-while loop.

```
#include <stdio.h>  
int main()  
{  
    int i = 1, j;  
    do  
    {  
        j = 1;  
        do  
        {  
            printf("%d ", j);  
            j++;  
        } while (j <= 3);  
        printf("\n");  
        i++;  
    } while (i <= 2);  
    return 0;  
}
```

```
}  
1 2 3  
1 2 3
```

Infinite Loop

Using for loop:

```
#include <stdio.h>  
int main () {  
    for ( ; ; ) {  
        printf("This loop will run forever.");  
    }  
    return 0;  
}  
This loop will run forever.  
This loop will run forever.  
This loop will run forever.  
...
```

Using While loop:

```
#include <stdio.h>  
int main() {  
    while (1)  
        printf("This loop will run forever.\n");  
    return 0;  
}  
This loop will run forever.  
This loop will run forever.  
This loop will run forever.  
...
```

Using the do-while loop:

```
#include <stdio.h>  
int main() {  
    do {  
        printf("This loop will run forever.");  
    } while (1);  
    return 0;  
}  
This loop will run forever.  
This loop will run forever.  
This loop will run forever.  
...
```

Jump or Loop Control Statements

Name	Description
------	-------------

Name	Description
break	The break statement is used to terminate the loop statement.
continue	When encountered, the continue statement skips the remaining body and jumps to the next iteration of the loop.
goto	goto statement transfers the control to the labeled statement.
return	Exit a function and optionally return a value

Break

The break statement is used to immediately exit a loop or switch statement.

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
            break;
        printf("%d ", i);
    }
    return 0;
}
1 2
```

Continue

The continue statement is used to **skip the current iteration** of a loop and move to the **next iteration**.

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
            continue;
        printf("%d ", i);
    }
    return 0;
}
1 2 4 5
```

goto

The goto statement is used to jump from one part of the program to another using a label.

```
#include <stdio.h>
```

```
int main()
{
    printf("Hello\n");
    goto end;
    printf("This will not be printed\n");
end:
    printf("End of program");
    return 0;
}
Hello
End of program
```

Return

The return statement is used to end a function and optionally send a value back to the calling function.

```
#include <stdio.h>
void display()
{
    printf("Hello\n");
    return;
    printf("Welcome");
}
```

```
int main()
{
    display();
    return 0;
}
Hello
```

```
#include <stdio.h>
int add()
{
    return 10 + 20;
}
int main()
{
    int result;
    result = add();
    printf("Sum = %d", result);
    return 0;
}
Sum = 30
```